KNN_Classification_And_Regression

August 10, 2025

1 K Nearest Neighbor Classifier Implementation

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,␣
↪classification_report
```

[2]:
```
X, y = make_classification(
        n_samples=1000,
        n_features=3,
        n_redundant=1,
        n_classes=2,
        random_state=999
)
```

[3]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,␣
↪random_state=42)
```

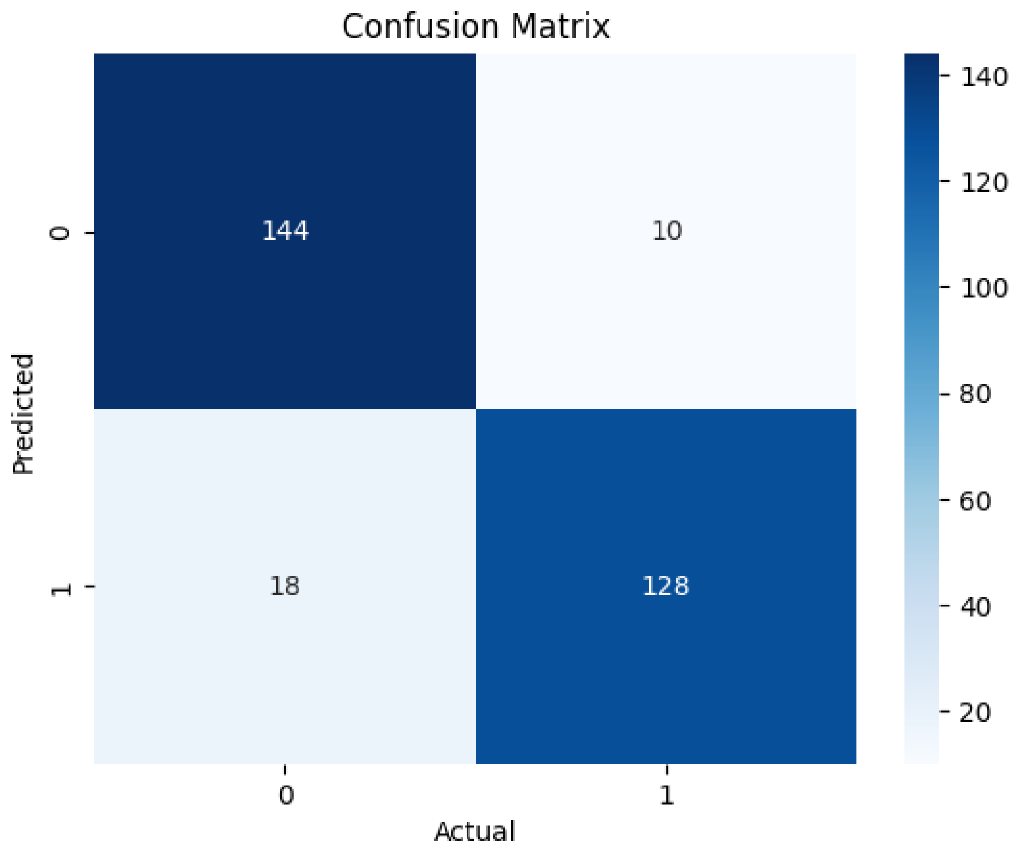
[4]:
```
knn = KNeighborsClassifier(n_neighbors=5, algorithm='auto', p=2)

# here in algorithms we have 4 inputs 1==> KDtree, 2==> Ball tree, 3==> Brute␣
↪force, 4==> auto(it is automatically select the best algorithms for the data)

# here p = 2 denotes the Euclidean distance and p = 1 denotes the manhattan␣
↪distance
```

[5]:
```
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

[6]:
```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues')
plt.xlabel('Actual')
```

```python
plt.ylabel('Predicted')
plt.title('Confusion Matrix')
```

[6]: Text(0.5, 1.0, 'Confusion Matrix')



[7]:
```python
print(accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
0.9066666666666666
              precision    recall  f1-score   support

           0       0.89      0.94      0.91       154
           1       0.93      0.88      0.90       146

    accuracy                           0.91       300
   macro avg       0.91      0.91      0.91       300
weighted avg       0.91      0.91      0.91       300
```

2 KNN Regressor

[8]:
```
from sklearn.datasets import make_regression
X, y = make_regression(
        n_samples=1000,
        n_features=2,
        noise=10,
        random_state=42
)
```

[9]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,␣
 ↪random_state=42)
```

[10]:
```
from sklearn.neighbors import KNeighborsRegressor
knnreg = KNeighborsRegressor(n_neighbors=6, algorithm='auto', p=1)
knnreg.fit(X_train, y_train)
```

[10]:
```
KNeighborsRegressor(n_neighbors=6, p=1)
```

[11]:
```
y_pred = knnreg.predict(X_test)
```

[12]:
```
from sklearn.metrics import mean_absolute_error, r2_score, mean_squared_error
print(mean_squared_error(y_test, y_pred))
print(mean_absolute_error(y_test, y_pred))
print('accuracy:', r2_score(y_test, y_pred)*100)
```

```
132.00564711851828
9.212948053616985
accuracy: 91.81055156910722
```